

Project 4: Heartbleed Lab

Tyler Vander Mooren

CYB 410: Secure Software

Dr. Linda Hamons

April 12, 2026

Project 4: Heartbleed Lab

Part 1 Research & Theory

The TLS heartbeat feature existed to allow one endpoint to verify that the other side of a TLS or Datagram TLS (DTLS) connection was still active without having to renegotiate the full session. The heartbeat extension provided a keep alive mechanism and supported DTLS for maximum transmission unit discovery. In a normal operation, one side would send a heartbeat request containing a payload and then the other side returns the heartbeat response containing an exact copy of that payload. The feature was intended to be lightweight and efficient rather than what it became which was complex and risky, which is part of why the implementation bug became so damaging in practice (Seggelmann et al., 2012). Heartbleed, formally identified as CVE-2014-0160, became one of the most significant software vulnerabilities in recent cybersecurity history because it exploited the TLS heartbeat feature in OpenSSL. OpenSSL is a cryptographic library used to protect web traffic, email, VPN connections and many other secure communications. While the vulnerability did not break the encryption mathematically it was still able to expose sensitive information because of a missing bounds check in the handling of the TLS heartbeat extension. If the exploit was successful it could leak sensitive information from process memory, this would include cryptographic keys and passwords (NIST, 2014).

The vulnerability worked because OpenSSL would trust an attacker-controlled length value in the heartbeat message instead of having it properly validated. If the payload length in a received heartbeat message is too large then the message must be discarded silently, however in the vulnerable versions of OpenSSL it failed to enforce this check correctly. In the official OpenSSL advisory, it states the bug was missing a “bounds check in the handling of the TLS heartbeat extension” which meant an attack could send a very small heartbeat request while

claiming it is a much larger payload. OpenSSL would then copy that and return the additional bytes from memory beyond the real payload, which creates an out-of-bounds read vulnerability (Seggelmann et al., 2012). What made Heartbleed especially serious is the type of information that could be exposed, the exploitation could leak information from memory locations which may contain highly sensitive information such as keys and passwords. It was also revealed that the vulnerability could expose up to 64 KB of memory to a connected client or server in a single request. Since process memory may contain active session cookies, usernames, keys, authentication material, user messages and other sensitive data; which creates a major confidentiality problem. Even though the vulnerability did not directly execute code or alter data, the fact it can repeatedly read memory made it extremely dangerous in real word systems (NIST, 2014).

Heartbleed teaches several valuable secure coding lessons. To begin, software must never trust external input, especially when concerning length and size fields without validating them against buffer boundaries. In Heartbleed, the core mistake was validation and it did not check if the claimed payload length actually fit inside the received record before copying memory. Also, secure systems require defensive programming and negative testing with malformed inputs in addition to the normal use cases. While a feature may appear simple and legitimate, if it parses attack-controlled data then it must be treated as a security boundary. Security assurance must also extend beyond just algorithms and protocols, reaching to the quality of the implementation itself. TLS was not fundamentally broken here; the failure was in how OpenSSL implemented the heartbeat behavior (Seggelmann et al., 2012). Another key lesson is that responding to a memory disclosure vulnerability requires more than installing a patch. The initial advice given to users was to upgrade to version 1.0.1g or if that was not possible to rebuild with heartbeat

support disabled. However, because of how exposed memory works it might have already included private keys, passwords or other credentials. So, organizations also needed to consider replacing keys, reissuing certificates and resetting compromised credentials. Once the confidentiality was lost, recovery required both software remediation and trust remediation. Heartbleed was a great example to show how incident response must also account for what may have been exposed, not just getting the vulnerable software updated as soon as possible (NIST, 2014). Heartbleed remains a defining example of how a small coding oversight can create a major cybersecurity incident. The TLS heartbeat feature was designed to provide an efficient keep-alive mechanism, but OpenSSL's failure to validate an attacker-supplied length field turned that feature into a memory disclosure vulnerability. The lesson learned is clear that strong security depends not only on sound protocols and cryptographic algorithms, but also on strict input validation, careful implementation, thorough testing and effective response when vulnerabilities are discovered (NIST, 2014; Seggelmann et al., 2012).

Part 2 Controlled Lab Setup

I attempted to complete the Heartbleed lab on Windows 11 but was unable to complete the vulnerable OpenSSL build successfully. Docker and WSL2 were installed and configured correctly. However, the legacy OpenSSL build repeatedly failed due to compatibility issues with the modern container environment. The picture attached shows this failure. A review of the build logs shows the failures occurring during the OpenSSL build and installation process, including a linker related failure and a documentation installation failure involving pod2man during later

installation attempts.

```
PS C:\Users\tyler\heartbleed-lab> notepad Dockerfile
PS C:\Users\tyler\heartbleed-lab> docker build -t heartbleed-vuln .
[+] Building 1.0s (6/9)                                docker:desktop-linux
=> [internal] load build definition from Dockerfile      0.0s
=> => transferring dockerfile: 440B                     0.0s
=> [internal] load metadata for docker.io/library/debian:7 0.6s
=> [auth] library/debian:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                        0.0s
=> => transferring context: 2B                          0.0s
=> CACHED [1/5] FROM docker.io/library/debian:7@sha256:2259b099d947443e44bbd1c94967c785361af8fd22df48a08a3942e2d 0.0s
=> => resolve docker.io/library/debian:7@sha256:2259b099d947443e44bbd1c94967c785361af8fd22df48a08a3942e2d5630849 0.0s
=> ERROR [2/5] RUN apt-get update && apt-get install -y build-essential wget git python python-pip openssl apache 0.3s
-----
> [2/5] RUN apt-get update && apt-get install -y build-essential wget git python python-pip openssl apache2:
0.281 E: Method http has died unexpectedly!
0.281 E: Sub-process http received a segmentation fault.
-----
Dockerfile:2
-----
1 | FROM debian:7
2 | >>> RUN apt-get update && apt-get install -y build-essential wget git python python-pip openssl apache2
3 | RUN wget https://www.openssl.org/source/old/1.0.1/openssl-1.0.1f.tar.gz && \
4 | tar -xvzf openssl-1.0.1f.tar.gz && cd openssl-1.0.1f && ./config && make && make install
-----
ERROR: failed to build: failed to solve: process "/bin/sh -c apt-get update && apt-get install -y build-essential wget git python python-pip openssl apache2" did not complete successfully: exit code: 100
PS C:\Users\tyler\heartbleed-lab>
```

Part 3 Secure Coding Reflection

In the original OpenSSL heartbeat code, the missing bounds check was a validation step to ensure that the claimed heartbeat payload length actually fit within the received TLS record. If the `payload_length` of a received `HeartbeatMessage` was too large, the message was supposed to be discarded silently. In the vulnerable code, this never happened and would just read the payload length field and use it without confirming. The code obviously failed because it trusted external input that should have been treated as untrusted. As a result, when the code was used with a false length value the response would include not only the legitimate payload but additional bytes from adjacent process memory. A correct fix must validate the minimum message size and then verify that the claimed payload length fits inside the actual record before copying any bytes. In other words, the code needs to reject malformed heartbeat messages instead of processing them. Below is a correct version in pseudocode.

```
int process_heartbeat(unsigned char *record, int record_len) {
    int payload_length;
    /* make sure the record is big enough to even be a heartbeat */
```

```
if (record_len < 19) {  
    return 0;  
}  
payload_length = (record[1] << 8) | record[2];  
/* if the payload length is bigger than what was really sent, reject it */  
if (payload_length + 19 > record_len) {  
    return 0;  
}  
/* otherwise process the heartbeat normally */  
return 1;  
}
```

The corrected version first checks that the heartbeat message is large enough to even be valid, then verifies that the claimed payload length actually fits inside the received record before copying or returning any data. If the length does not match the size of the message then the heartbeat should just be rejected instead of processed.

References

- National Institute of Standards and Technology (NIST) (2014). CVE-2014-0160. National Vulnerability Database. nvd.nist.gov/vuln/detail/cve-2014-0160
- Seggelmann, R., Tuexen, M., & Williams, M. (2012). *Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS) heartbeat extension* (RFC 6520).
<https://doi.org/10.17487/RFC6520>